VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Intelligent Fashion E-Commerce System with AI Chatbot and Scalable Architecture

By
Nguyen Hoang Giang
ITITIU21192

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Information Technology / Computer Science / Computer Engineering

Ho Chi Minh City, Vietnam
Year 2025

Intelligent Fashion E-Commerce System with AI Chatbot and Scalable Architecture

APPROVED BY:

_____

Huynh Kha Tu, Assoc. Prof

THESIS COMMITTEE

_____

Committee Chair Name, Title

Committee Chair

_____

Committee Member Name, Title

Member

_____

Secretary Member Name, Title

Secretary Member

Acknowledgments

I would like to express my sincere gratitude to my supervisor, faculty members, teammates, and family for their continuous support throughout this thesis project.

Contents

Acknowledgments **2**

List of Figures **6**

List of Tables **7**

List of Abbreviations **8**

Abstract **9**

1 INTRODUCTION **10**
1.1 Background . . . 10
1.2 Problem Statement . . . 10
1.3 Scope . . . 10
1.3.1 In Scope . . . 10
1.3.2 Out of Scope . . . 11
1.4 Objectives . . . 11
1.5 Assumptions and Proposed Approach . . . 11
1.5.1 Assumptions . . . 11
1.5.2 Proposed Approach . . . 12
1.6 Main Contributions . . . 12
1.7 Thesis Structure . . . 12

2 LITERATURE REVIEW/RELATED WORK **13**
2.1 Introduction . . . 13
2.2 E-Commerce Backend Architectures . . . 13
2.3 Authentication and Session Security . . . 13
2.4 Catalog, Cart, and Transaction Consistency . . . 13
2.5 Checkout Reservation and Concurrency Control . . . 14
2.6 Payment Callback Reliability . . . 14
2.7 Real-Time Customer Support Systems . . . 14
2.8 Scalability and Operational Reliability . . . 14
2.9 Research Gap and Thesis Positioning . . . 15

3 METHODOLOGY **16**
3.1 Methodological Approach . . . 16
3.1.1 Design Process . . . 16
3.2 Backend Requirement Specification . . . 17
3.2.1 Functional Requirements . . . 17
3.2.2 Non-Functional Requirements . . . 17

3.3 System Design . . . 17
3.3.1 Overall Backend Architecture . . . 17
3.3.2 Data Design and Entity Relationships . . . 18
3.3.3 Use Case Boundary . . . 19
3.4 Workflow Modeling for Core Features . . . 20
3.4.1 Authentication and Account Recovery Workflow . . . 20
3.4.2 Catalog, Cart, and Checkout Workflow . . . 23
3.4.3 Payment Finalization Workflow . . . 24
3.4.4 AI Chat and Human Handoff Workflow . . . 25
3.4.5 Reservation Expiry and Staff Reassignment Workflow . . . 26
3.5 Evaluation Methodology . . . 27
3.5.1 Evaluation Scenarios . . . 27
3.5.2 Evaluation Metrics . . . 27
3.6 Chapter Summary . . . 28

4 IMPLEMENTATION 29
4.1 Implementation Environment . . . 29
4.1.1 Technology Stack . . . 29
4.1.2 Runtime Initialization . . . 29
4.2 Backend Module Organization . . . 29
4.3 Core Feature Implementation . . . 30
4.3.1 Feature 1: Authentication and Session Security . . . 30
4.3.2 Feature 2: Product Catalog and Query Filtering . . . 31
4.3.3 Feature 3: Cart Lifecycle Management . . . 31
4.3.4 Feature 4: Checkout and Reservation Engine . . . 31
4.3.5 Feature 5: Payment Callback and Order Finalization . . . 32
4.3.6 Feature 6: Real-Time AI Chat and Human Handoff . . . 32
4.4 Cross-Cutting Engineering Concerns . . . 32
4.4.1 Validation, Security, and Error Boundaries . . . 32
4.4.2 Observability and Operational Safety . . . 33
4.5 Chapter Summary . . . 33

5 DISCUSSION AND EVALUATION 34
5.1 Evaluation Scope . . . 34
5.1.1 Evaluation Targets . . . 34
5.1.2 Evaluation Strategy . . . 34
5.2 Scenario-Based Results . . . 34
5.2.1 Authentication and Session Security . . . 34
5.2.2 Catalog and Cart Behavior . . . 35
5.2.3 Checkout Reservation Consistency . . . 35
5.2.4 Payment Callback Finalization . . . 36
5.2.5 Realtime Chat and Handoff Continuity . . . 36
5.3 Discussion of Engineering Trade-offs . . . 37
5.3.1 Modular Monolith vs. Distributed Services . . . 37
5.3.2 Redis Coordination vs. Pure Database Locking . . . 37
5.3.3 AI-first Support vs. Human-only Support . . . 37
5.4 Limitations and Threats to Validity . . . 37
5.5 Chapter Summary . . . 38

6 CONCLUSION AND FUTURE WORK **39**
6.1 Conclusion 39
6.2 Achievement of Objectives 39
6.3 Limitations 40
6.4 Future Work 40
6.5 Closing Remarks 40

A Additional Technical Materials **42**
A.1 API Endpoint Summary 42
A.2 Redis Key Namespace 42
A.3 Additional Sequence Diagrams 42
A.4 Configuration Snapshot 43

List of Figures

3.1 Overall Backend Architecture 18
3.2 Core Backend Schema Diagram 19
3.3 Use Case Coverage for Backend Interactions 20
3.4 Authentication Lifecycle Workflow 21
3.5 Registration and Verification Workflow 22
3.6 Password Recovery Workflow 23
3.7 Catalog-to-Cart Workflow 24
3.8 Checkout-to-Payment Finalization Workflow 25
3.9 AI Chat and Human Handoff Workflow 26
3.10 Reservation Expiry and Staff Reassignment Workflow 27

4.1 Backend Module Structure in Source Code 30
4.2 Authentication API Implementation Evidence 30
4.3 Checkout Reservation State Evidence 31
4.4 Payment Callback and Finalization Evidence 32
4.5 AI Chat to Human Handoff Implementation Evidence 32

List of Tables

3.1 Core Functional Requirements 17
3.2 Non-Functional Requirements and Targets 17
3.3 Evaluation Metrics by Concern 27

4.1 Backend Module Structure 30

5.1 Authentication Evaluation Outcomes 35
5.2 Catalog and Cart Evaluation Outcomes 35
5.3 Checkout Reservation Evaluation Outcomes 36
5.4 Payment Finalization Evaluation Outcomes 36
5.5 Realtime Chat Evaluation Outcomes 37

A.1 Redis Key Patterns Used in the Backend 42

List of Abbreviations

ABC - Example Abbreviation

Abstract

The rapid growth of online fashion retail has increased the demand for intelligent, responsive, and scalable e-commerce systems. This project proposes the development of an Intelligent Fashion E-Commerce System that integrates an AI-powered chatbot capable of handling real-time customer support and automatically escalating complex queries to human staff through socket-based communication. The chatbot is designed to resolve common inquiries such as order tracking, product details, and return policies, while unresolved issues are redirected to staff who can view the entire conversation history and provide instant assistance.

To ensure stable performance during peak traffic periods, particularly in sale seasons, the system adopts scalable techniques using third-party tools such as Redis, RabbitMQ, or Kafka, allowing the monolithic backend to efficiently manage a large number of concurrent client requests. The proposed system aims to enhance user experience, improve customer support efficiency, and demonstrate an effective method for scaling monolithic architectures in high-demand fashion e-commerce environments. The project will be implemented using modern web technologies, including Node.js, Socket.io, and React.js, and evaluated based on performance, scalability, and user satisfaction.

Keywords: fashion e-commerce, AI chatbot, real-time customer support, scalability, monolithic architecture

Chapter 1

INTRODUCTION

1.1 Background

The rapid growth of fashion e-commerce has increased competition among online retailers and raised customer expectations for fast, reliable, and personalized services. Traditional e-commerce systems often rely on manual customer support and synchronous processing patterns that are difficult to scale during high-traffic periods such as seasonal campaigns and flash sales.

Recent advances in AI-assisted customer support and real-time communication provide an opportunity to improve service responsiveness. However, practical deployment in production still requires strong backend engineering, especially for authentication security, checkout consistency, payment callback correctness, and resilient customer support workflows.

1.2 Problem Statement

Despite market growth, many fashion e-commerce platforms still face backend limitations in three critical areas. First, customer support workflows are often inefficient: chatbot-only systems fail on complex cases, while manual-only support does not scale. Second, transactional consistency during checkout is vulnerable to race conditions when stock and discount quotas are accessed concurrently. Third, payment finalization and callback handling can become unreliable without robust verification and idempotent processing.

These gaps motivate the need for a backend architecture that combines secure API design, concurrency-safe reservation control, and real-time support escalation.

1.3 Scope

1.3.1 In Scope

This thesis focuses on backend design and implementation for the following modules:

- Authentication and session security: register, email verification, login, refresh token, logout, and password reset.
- Product catalog APIs and query filtering for product discovery.
- Cart lifecycle management with stock-aware validation and cart state control.

- Checkout reservation engine using Redis for stock and voucher reservation under concurrency.
- Payment callback handling with signature verification and order finalization.
- Real-time AI chat with human handoff using Socket.IO, staff availability, and reassignment logic.

1.3.2 Out of Scope

The following areas are not the primary focus of this thesis:

- Frontend UI/UX design and usability experiments.
- Native mobile application development.
- Training custom large language models.
- Full cloud-native microservices migration and large-scale production autoscaling.

1.4 Objectives

The project objectives are:

- Build a modular backend architecture for a fashion e-commerce platform with clear separation of API, service, and data layers.
- Implement a consistent checkout flow with concurrency-safe reservation for stock and voucher resources.
- Integrate payment callback processing with request verification and idempotent order updates.
- Implement AI-assisted chat support with reliable human escalation and real-time session continuity.
- Evaluate the backend in terms of performance, reliability, and maintainability.

1.5 Assumptions and Proposed Approach

1.5.1 Assumptions

- Users have stable network connectivity and standard web-client behavior.
- External providers (AI and payment gateway) are reachable and respond within acceptable latency bounds.
- Redis and PostgreSQL services are available for coordination and transactional persistence.

1.5.2 Proposed Approach

- Use a modular monolithic backend architecture on Node.js and Express.
- Keep PostgreSQL as the source of truth for domain and transactional data.
- Use Redis for short-lived coordination state such as reservations, chat state, and heartbeats.
- Use Socket.IO for real-time support communication and handoff orchestration.
- Use explicit validation, error boundaries, and signature verification to improve operational safety.

1.6 Main Contributions

This thesis provides the following technical contributions:

- A backend module architecture that separates routing, orchestration, business logic, and persistence for maintainable development.
- An atomic reservation strategy for stock and voucher quotas using Redis keys and Lua-based coordination.
- A payment finalization flow with callback signature validation and idempotent processing safeguards.
- A real-time AI chat workflow with human escalation, staff heartbeat monitoring, and reassignment handling.

1.7 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 reviews related work in e-commerce backend systems, chatbot support, real-time communication, and scalability techniques.
- Chapter 3 presents the methodology, including architecture design, data model design, and workflow modeling.
- Chapter 4 describes implementation details for the six core backend features.
- Chapter 5 discusses evaluation results, design trade-offs, and system limitations.
- Chapter 6 concludes the thesis and outlines directions for future work.

Chapter 2

LITERATURE REVIEW/RELATED WORK

2.1 Introduction

This chapter reviews technical foundations and prior approaches relevant to the backend scope of this thesis. The review focuses on architecture design, authentication and session control, transactional consistency in checkout, payment callback reliability, real-time support workflows, and scalability strategies.

2.2 E-Commerce Backend Architectures

Backend systems for e-commerce commonly adopt either modular monolithic architecture or microservices architecture. Microservices provide independent deployment and scaling at service boundaries, but they also increase distributed-system complexity, operational overhead, and cross-service consistency risks.

For projects with constrained team size and delivery timeline, modular monoliths remain practical when internal boundaries are clear. The route-controller-service-repository layering pattern is frequently used to maintain code organization while keeping deployment simple.

2.3 Authentication and Session Security

Authentication in modern web systems usually combines short-lived access tokens with longer-lived refresh tokens. Additional protection mechanisms include email verification, password reset flows, and session revocation.

Common challenges include token theft impact, stale session accumulation, and weak recovery flows. Prior work emphasizes strict validation at API boundaries and explicit session lifecycle tracking for better operational security.

2.4 Catalog, Cart, and Transaction Consistency

Catalog and cart modules require consistent stock visibility and predictable behavior under concurrent updates. Typical issues include stale availability information, race conditions when updating cart quantities, and conflicts during checkout transition.

Design approaches in the literature include pessimistic database locks, optimistic control with version checks, and hybrid patterns that combine persistent data with short-lived coordination state.

2.5 Checkout Reservation and Concurrency Control

Checkout consistency becomes difficult when multiple users attempt to reserve the same stock or voucher quotas at nearly the same time. Approaches discussed in prior systems include:

- direct database locking strategies;
- distributed in-memory reservation keys;
- atomic scripts to prevent partial resource updates.

In-memory coordination layers such as Redis are widely used for reservation windows and expiration-driven rollback behavior in high-traffic purchasing flows.

2.6 Payment Callback Reliability

Payment finalization in e-commerce often relies on asynchronous callbacks from external gateways. Reliable handling requires three controls:

- signature verification to authenticate callback origin;
- idempotent processing to prevent duplicate state transitions;
- explicit synchronization between payment status and order status.

Without these controls, systems risk inconsistent order states and financial reconciliation errors.

2.7 Real-Time Customer Support Systems

AI-assisted chat support is increasingly used to reduce response time for repetitive requests. However, purely automated chat is often insufficient for complex customer issues. Many production systems therefore adopt a hybrid model: AI for first-response handling and human escalation for unresolved cases.

Socket-based communication patterns are common for live support channels, especially when workload distribution and reassignment are required for support staff continuity.

2.8 Scalability and Operational Reliability

Scalability practices in backend systems include caching, transient coordination data, asynchronous task execution, and health monitoring. Redis is frequently used for short-lived state (for example, reservation counters, heartbeat keys, and chat session metadata).

Operational reliability is also shaped by error isolation, structured logging, and graceful shutdown behavior across API servers, database connections, and realtime channels.

2.9 Research Gap and Thesis Positioning

Prior approaches usually optimize one or two concerns in isolation, such as authentication, payment correctness, or chatbot automation. Fewer works provide a unified backend design that addresses session security, checkout concurrency control, payment callback reliability, and realtime human-handoff support in one consistent implementation.

This thesis is positioned to address that integration gap through a backend architecture that combines modular API design, Redis-assisted coordination, and realtime support continuity mechanisms.

Chapter 3

METHODOLOGY

3.1 Methodological Approach

This thesis follows a system design-and-implementation methodology focused on backend engineering. The method combines requirement analysis, architecture design, workflow modeling, and evaluation planning before detailed implementation.

The objective of this chapter is to define how the backend is designed to satisfy practical requirements for security, consistency, scalability, and real-time support continuity.

3.1.1 Design Process

The design process is executed in five stages:

- Stage 1: derive requirements from target user flows (account, cart, checkout, payment, support).
- Stage 2: define module boundaries for API, business logic, and persistence.
- Stage 3: model data and transaction-critical workflows.
- Stage 4: model real-time support workflows with escalation and reassignment.
- Stage 5: define evaluation criteria for performance and reliability.

3.2 Backend Requirement Specification

3.2.1 Functional Requirements

Table 3.1: Core Functional Requirements

ID	Requirement	Design Intent
FR1	Authentication and session lifecycle management	Secure login state, token rotation, and session revocation
FR2	Product catalog and filtering APIs	Fast product discovery with stock-aware payloads
FR3	Cart lifecycle operations	Correct add/update/remove behavior with stock validation
FR4	Checkout reservation control	Atomic reservation for stock and voucher quotas
FR5	Payment callback finalization	Verified callback processing and idempotent order updates
FR6	Real-time AI chat with human handoff	Continuous support flow with staff assignment and fallback

3.2.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements and Targets

Category	Target	Design Strategy
Performance	Stable API latency under concurrent load	Redis-assisted coordination and optimized service boundaries
Consistency	No over-reservation of stock and vouchers	Atomic Redis/Lua reservation and TTL-based rollback
Security	Protected access and trusted callbacks	JWT validation, session control, signature verification
Reliability	Safe recovery from partial failures	Idempotent payment handling and explicit error boundaries
Maintainability	Clear module ownership	Route-controller-service-repository layering

3.3 System Design

3.3.1 Overall Backend Architecture

The backend uses a modular monolithic architecture with Express for HTTP APIs and Socket.IO for realtime channels. PostgreSQL is the transactional source of truth, while

Redis is used for short-lived coordination state such as reservation counters, chat session metadata, and heartbeat TTL keys.

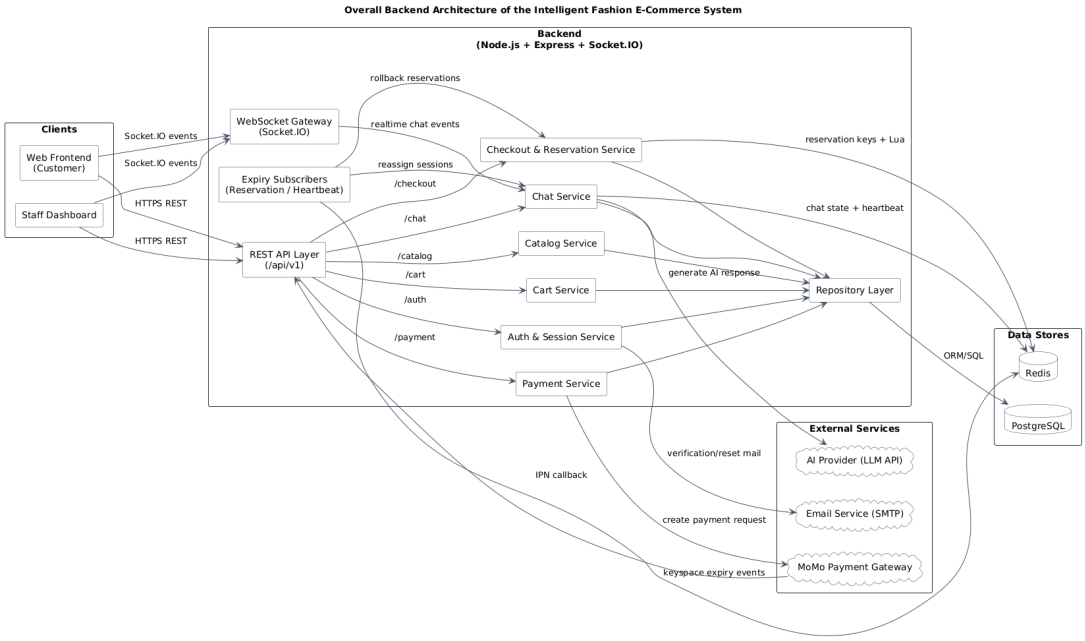

Figure 3.1: Overall Backend Architecture

3.3.2 Data Design and Entity Relationships

The persistence model captures account, catalog, cart, order, discount, and chat domains. Entity relationships are designed to support both transactional operations (checkout and payment) and conversational support workflows (sessions, messages, transfers).

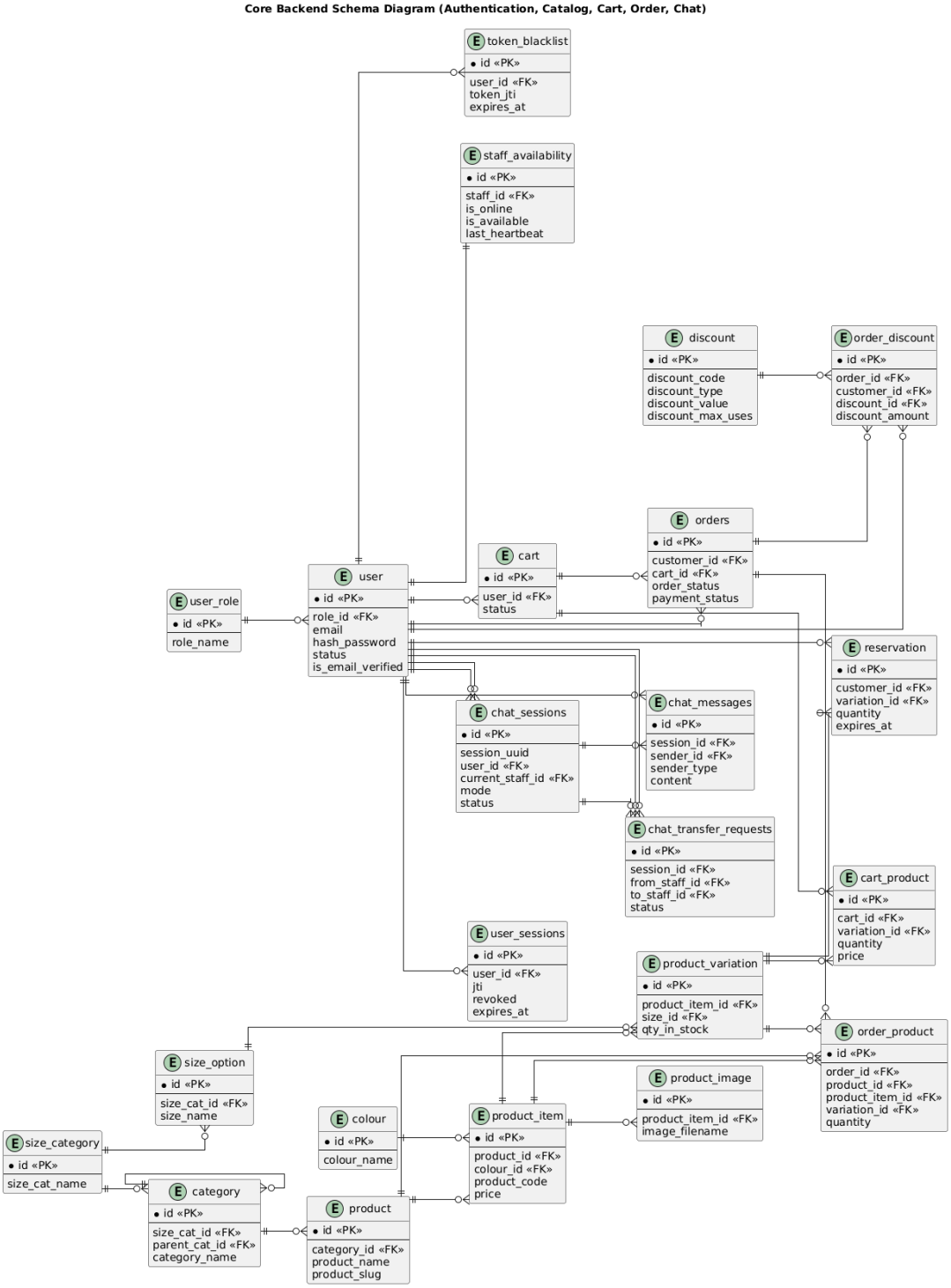

Figure 3.2: Core Backend Schema Diagram

3.3.3 Use Case Boundary

Use case modeling is used to define external actor interactions with the backend, including customer actions, support staff operations, and payment provider callbacks.

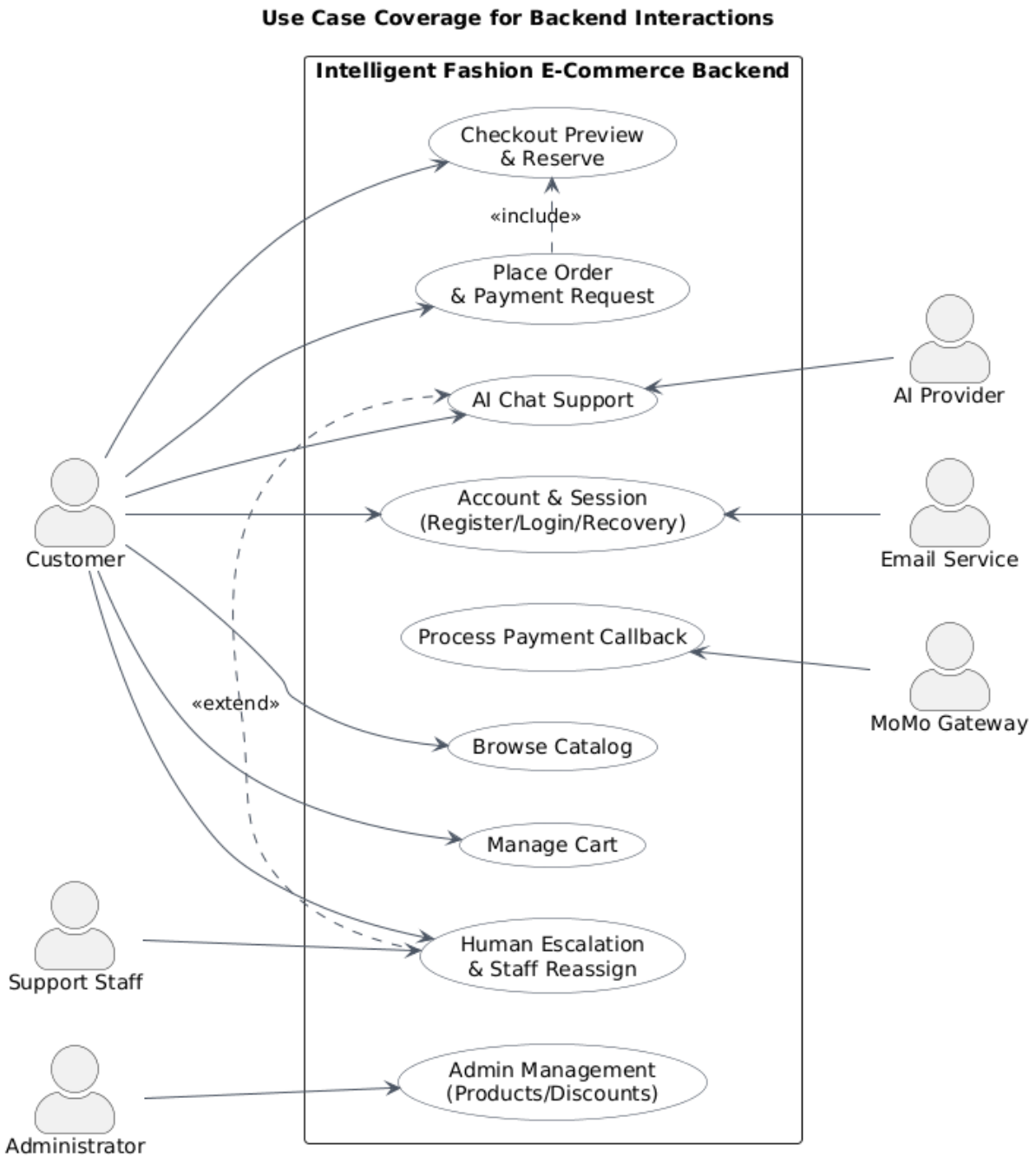

Figure 3.3: Use Case Coverage for Backend Interactions

3.4 Workflow Modeling for Core Features

3.4.1 Authentication and Account Recovery Workflow

This workflow includes registration, email verification, login, refresh token, logout, and password recovery. The model emphasizes token lifecycle control and secure account restoration.

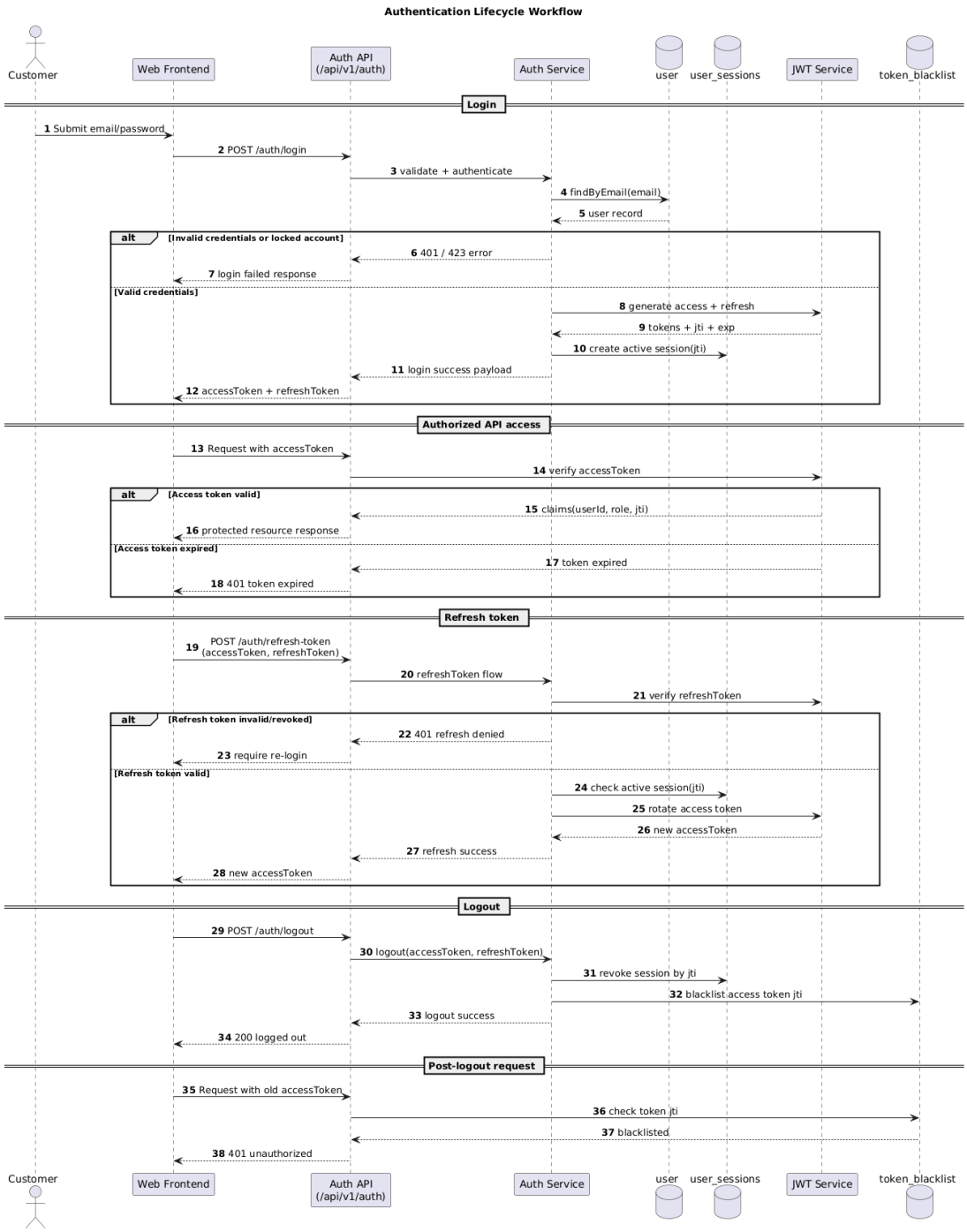

Figure 3.4: Authentication Lifecycle Workflow

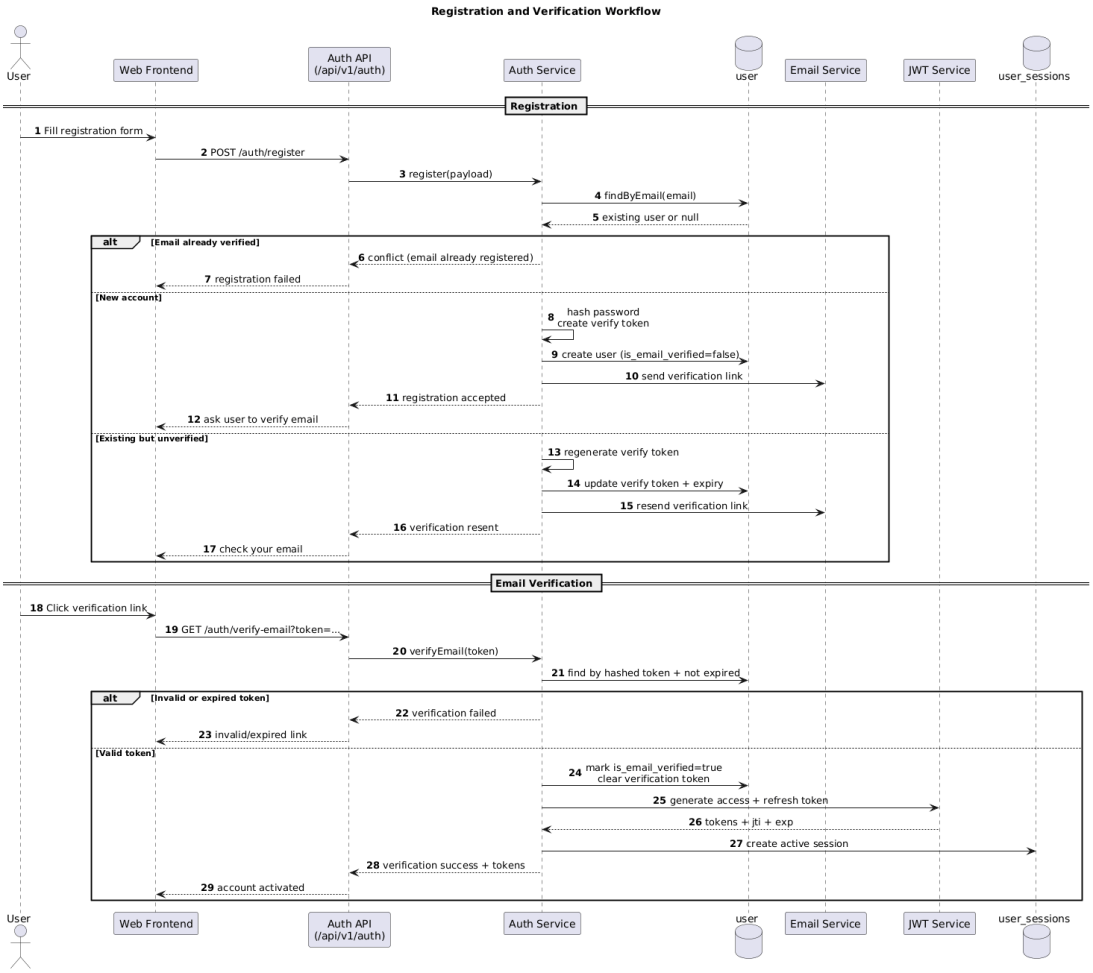

Figure 3.5: Registration and Verification Workflow

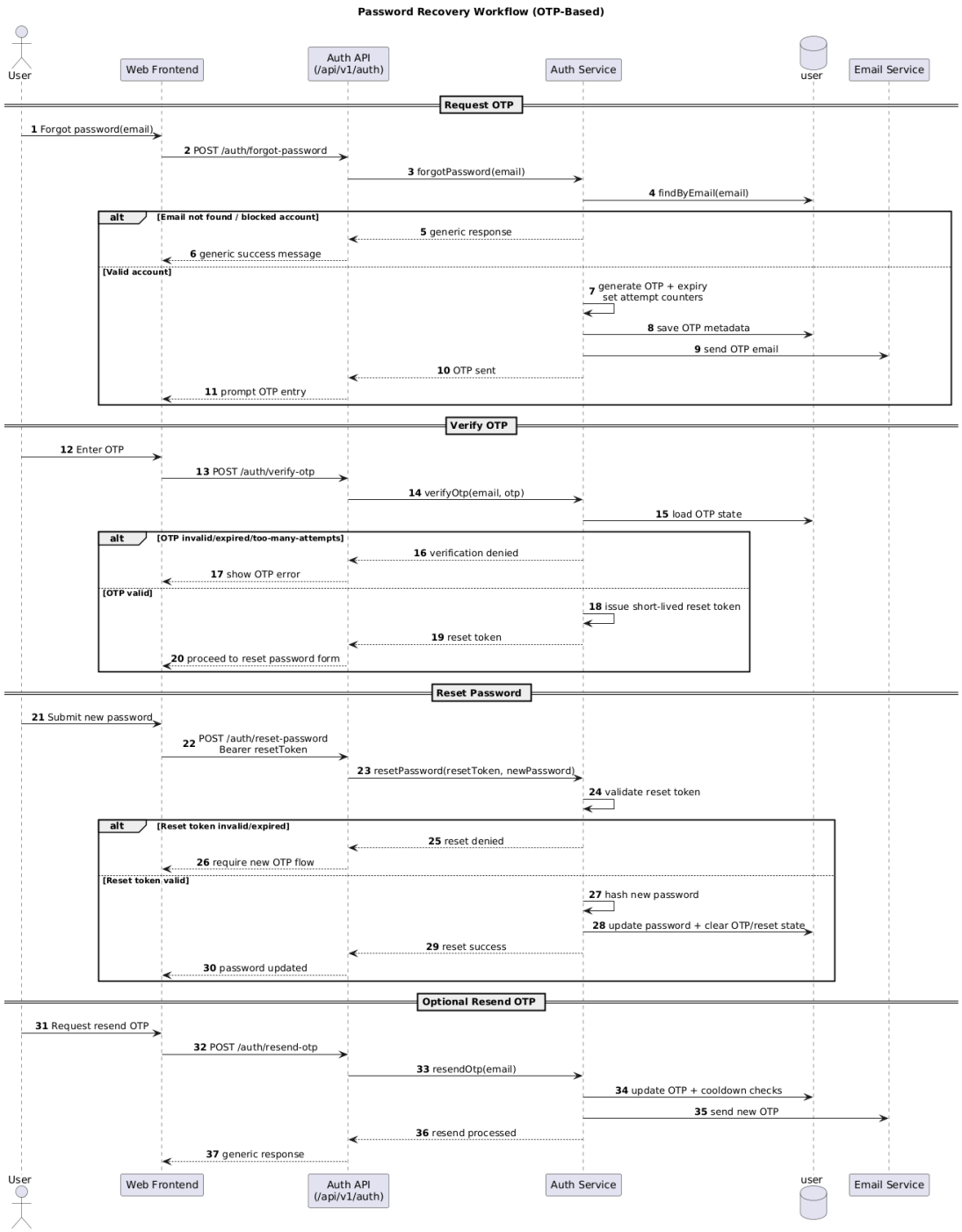

Figure 3.6: Password Recovery Workflow

3.4.2 Catalog, Cart, and Checkout Workflow

This workflow models product selection, cart mutation, checkout preview, and reservation setup. It focuses on preventing race conditions during high-concurrency purchase attempts.

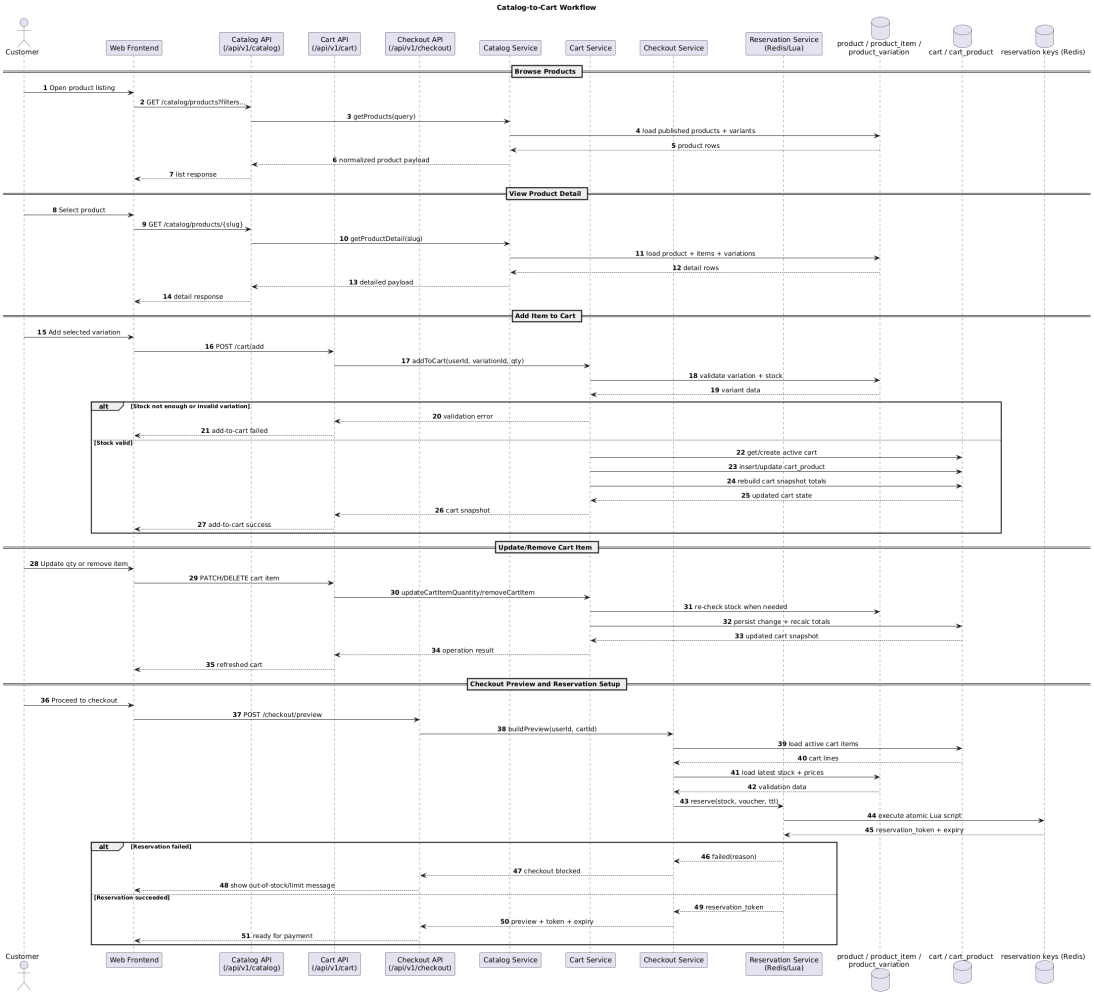

Figure 3.7: Catalog-to-Cart Workflow

3.4.3 Payment Finalization Workflow

This workflow captures callback verification, order-state reconciliation, and idempotent finalization.

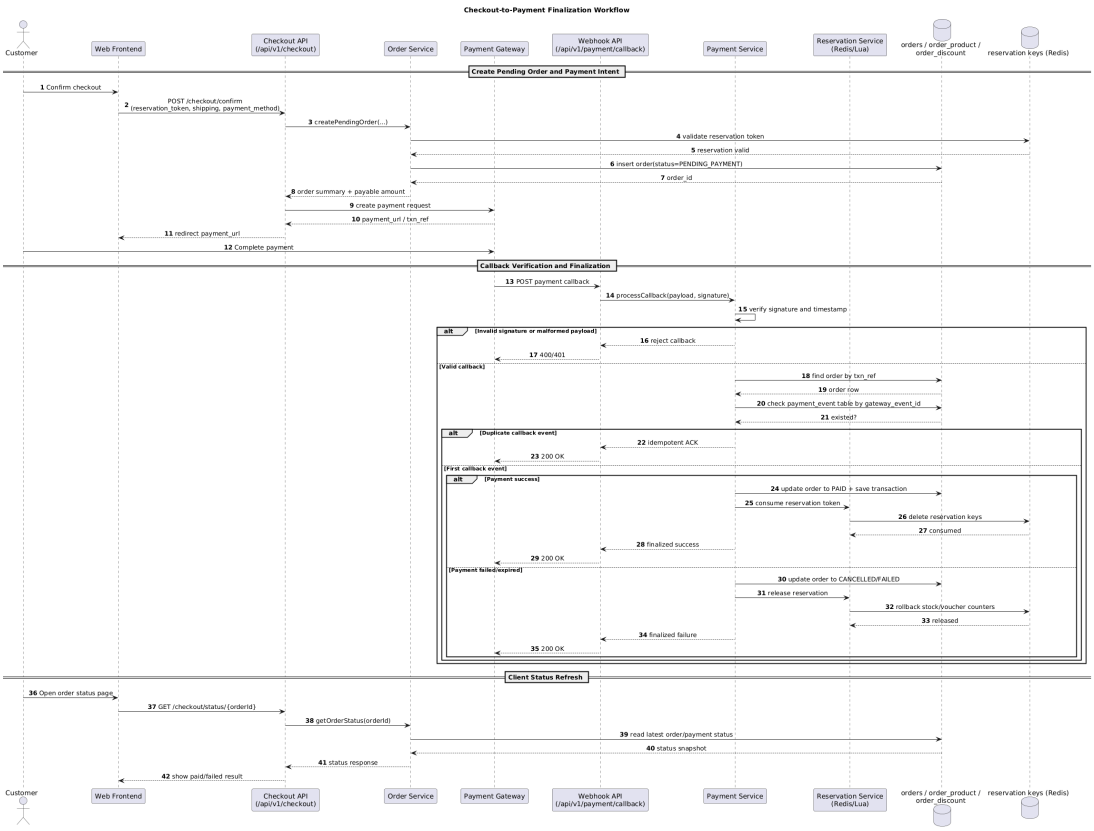

Figure 3.8: Checkout-to-Payment Finalization Workflow

3.4.4 AI Chat and Human Handoff Workflow

This workflow models realtime conversation handling, escalation to staff, and continuity mechanisms when staff availability changes.

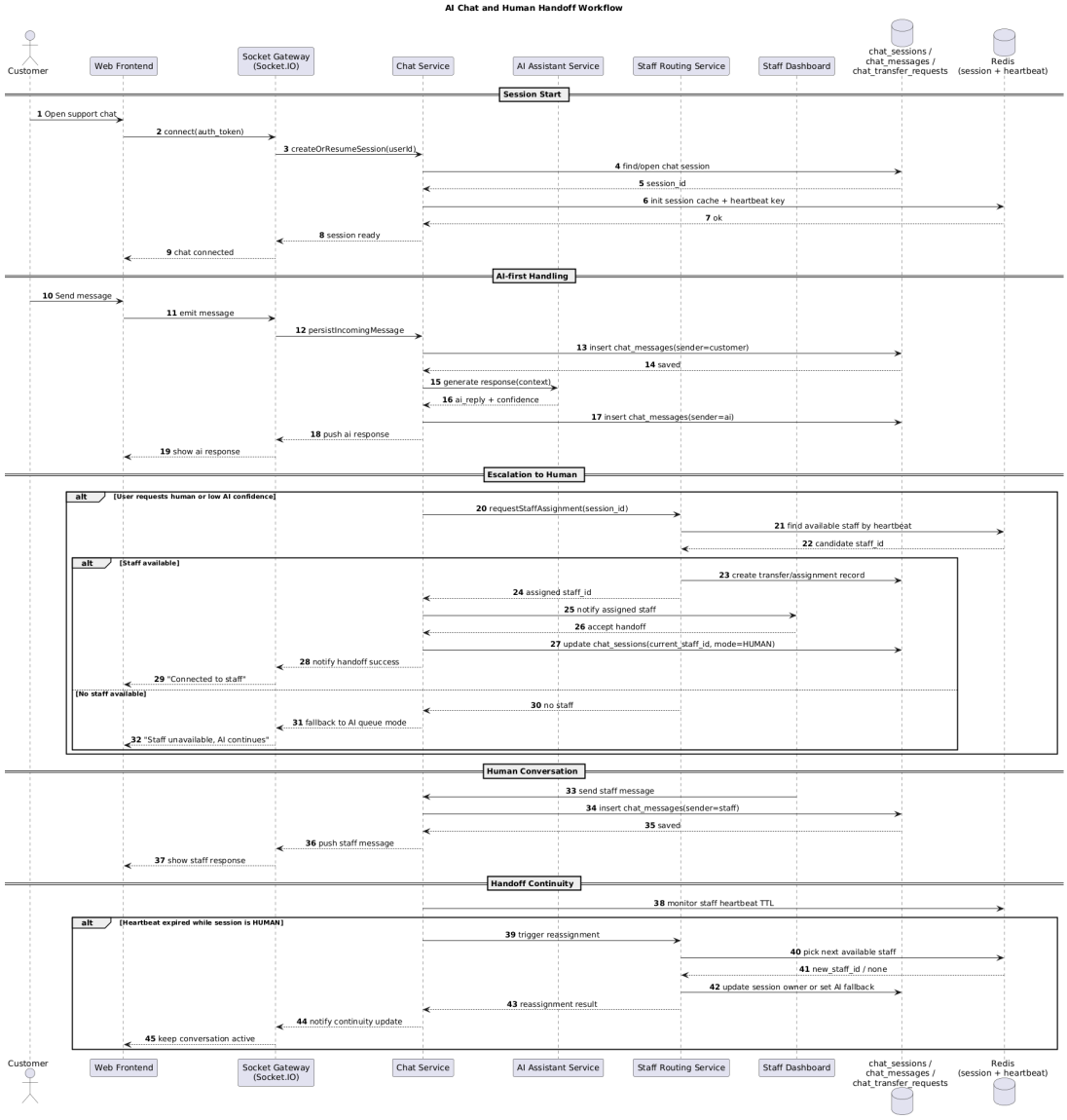

Figure 3.9: AI Chat and Human Handoff Workflow

3.4.5 Reservation Expiry and Staff Reassignment Workflow

Two reliability workflows are modeled in this thesis implementation:

- Reservation expiry rollback for stale pending orders.
- Staff heartbeat expiry and session reassignment for support continuity.

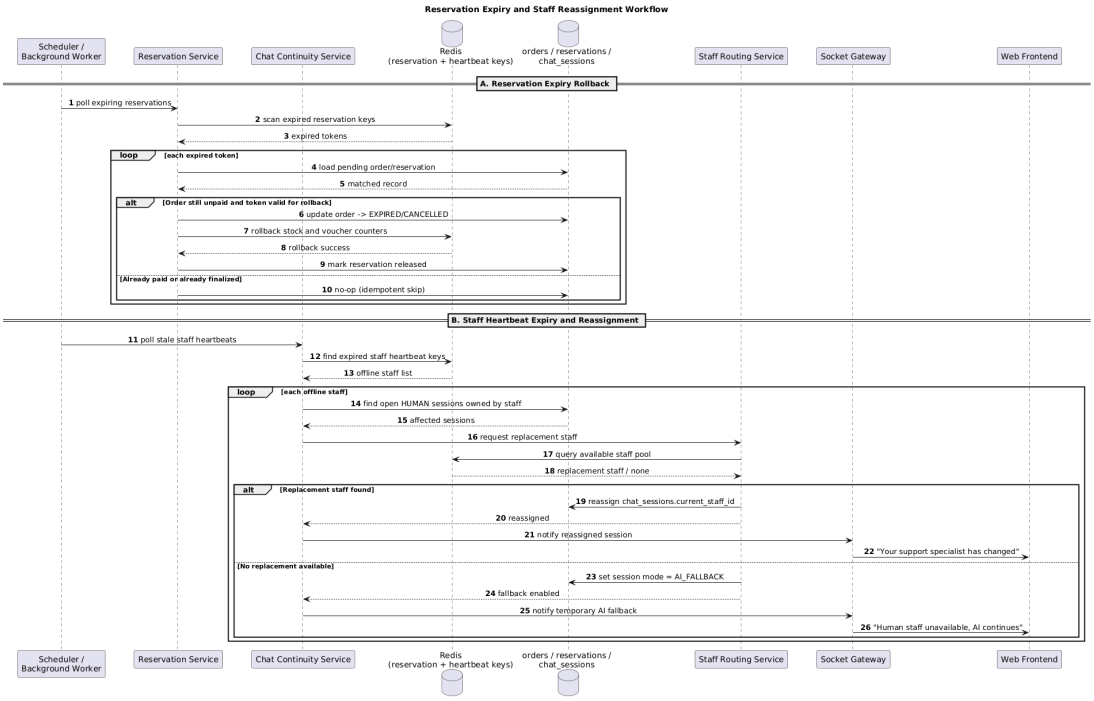

Figure 3.10: Reservation Expiry and Staff Reassignment Workflow

3.5 Evaluation Methodology

3.5.1 Evaluation Scenarios

The system is evaluated using scenario-based testing:

- Standard user purchase flow from login to paid order.
- Concurrent checkout on limited stock and discount quotas.
- Delayed and repeated payment callback handling.
- AI chat escalation and reassignment when staff disconnects.

3.5.2 Evaluation Metrics

Table 3.3: Evaluation Metrics by Concern

Concern	Metric Examples
API performance	p50/p95 latency, throughput, timeout rate
Transactional reliability	reservation rollback success rate, duplicate callback safety
Realtime support	chat response delay, handoff delay, reassignment delay
Security correctness	invalid token rejection, invalid signature rejection

3.6 Chapter Summary

This chapter defines the methodology used to design the backend system and to model critical workflows before implementation. The next chapter presents how these designs are realized in the actual backend modules and services.

Chapter 4

IMPLEMENTATION

4.1 Implementation Environment

4.1.1 Technology Stack

- Runtime and API framework: Node.js (ESM) with Express.
- Persistence: PostgreSQL with Sequelize ORM.
- Coordination and short-lived state: Redis.
- Realtime communication: Socket.IO.
- Payment integration: MoMo payment API (create payment and IPN callback).
- AI integration: provider-based AI service (OpenAI/OpenRouter via config).

4.1.2 Runtime Initialization

- Server entrypoint initializes PostgreSQL, Redis client, HTTP server, and Socket.IO.
- Redis expiry subscribers are started for reservation rollback and staff heartbeat timeout handling.
- Environment configuration is validated by Zod schema before startup.
- Graceful shutdown closes HTTP server, websocket server, subscribers, Redis, and PostgreSQL connection.

4.2 Backend Module Organization

The backend follows a layered modular structure.

Table 4.1: Backend Module Structure

Module	Responsibility
`routes/`	Define HTTP endpoints and bind middleware/controller handlers.
`controllers/`	Parse request context, call services, map responses.
`services/`	Core business logic and cross-service orchestration.
`reponsitories/`	Data access abstraction on top of Sequelize models.
`models/`	Database schema mapping and entity relations.
`validations/`	Zod-based request payload schema validation.
`middlewares/`	Auth, validation, and HTTP boundary concerns.
`websocket/`	Socket.IO auth and event handlers for chat and staff workflow.

API routes are grouped under `/api/v1`: `auth`, `catalog`, `cart`, `checkout`, `payment`, `chat`, and `redis` (debug).

Placeholder for module structure image
Suggested file: `figures/ch4_module_structure.png`

Figure 4.1: Backend Module Structure in Source Code

4.3 Core Feature Implementation

4.3.1 Feature 1: Authentication and Session Security

Implemented in `auth.route.js`, `auth.controller.js`, and `auth.service.js`.

- Registration and verification: `POST /auth/register`, `GET /auth/verify-email`, `POST /auth/resend-verification`.
- Login and token lifecycle: `POST /auth/login`, `POST /auth/refresh-token`, `POST /auth/logout`.
- Password recovery: `POST /auth/forgot-password`, `POST /auth/verify-otp`, `POST /auth/reset-password`, `POST /auth/resend-otp`.
- Security controls include bcrypt hashing, access/refresh token separation, and session invalidation logic.

Placeholder for authentication API evidence
Suggested file: `figures/ch4_auth_api_evidence.png`

Figure 4.2: Authentication API Implementation Evidence

4.3.2 Feature 2: Product Catalog and Query Filtering

Implemented in `catalog.route.js` and `catalog.service.js`.

- `GET /catalog/products`: paginated listing with filter and sorting support.
- `GET /catalog/products/:slug`: detailed product response by slug.
- `GET /catalog/filters`: filter metadata for frontend querying.
- Catalog service maps relational product data into API response payloads for list and detail views.

4.3.3 Feature 3: Cart Lifecycle Management

Implemented in `cart.route.js`, `cart.controller.js`, and `cart.service.js`.

- `POST /cart/add`: validates variation and stock before append/update.
- `GET /cart`: returns authenticated user's active cart snapshot.
- `PATCH /cart/items/:cartItemId`: quantity update with stock recheck.
- `DELETE /cart/items/:cartItemId`: remove item from active cart.
- Service guards include user existence check, active-cart resolution, cart lock protection, and snapshot recomputation.

4.3.4 Feature 4: Checkout and Reservation Engine

Implemented in `checkout.route.js` and `checkout.service.js`.

- `GET /checkout/preview`: validates cart items, computes subtotal/discount, and checks reservation capacity.
- `POST /checkout/place-order`: creates pending order and payment intent after atomic reservation.
- `GET /checkout/order-status/:orderId`: polls payment and order state.
- Redis Lua script performs atomic reservation for stock and voucher counters.
- Reservation metadata is cached to support deterministic rollback and post-payment reconciliation.

Placeholder for reservation evidence (Redis keys / checkout snapshot)
Suggested file: `figures/ch4_checkout_reservation_evidence.png`

Figure 4.3: Checkout Reservation State Evidence

4.3.5 Feature 5: Payment Callback and Order Finalization

Implemented in `payment.route.js`, `payment.controller.js`, and `payment.service.js`.

- `POST /payment/momo/ipn`: receives MoMo callback and verifies signature.
- `GET /payment/momo/redirect`: redirects return params to frontend result route.
- Finalization flow updates order state, persists order items/discounts, adjusts stock, and clears purchased cart items.
- Callback handling applies idempotent processing strategy for repeated notifications.

Placeholder for payment callback evidence (IPN logs/result)
Suggested file: `figures/ch4_payment_ipn_evidence.png`

Figure 4.4: Payment Callback and Finalization Evidence

4.3.6 Feature 6: Real-Time AI Chat and Human Handoff

Implemented across `chat.route.js`, `chat.service.js`, `staff.service.js`, and `websocket/`.

- Session operations: `GET /chat/session/active`, `GET /chat/session/:sessionUuid/histor`
  `POST /chat/session/:sessionUuid/request-human`, `POST /chat/session/:sessionUuid/`
- Staff operations: `POST /chat/staff/availability`, `GET /chat/staff/workload`.
- Socket middleware authenticates users and joins role-based rooms.
- Chat service supports AI-first replies, intent routing, and escalation to human support.
- Staff service handles assignment, heartbeat monitoring, disconnect recovery, and reassignment.

Placeholder for chat handoff evidence (assignment/reassignment events)
Suggested file: `figures/ch4_chat_handoff_evidence.png`

Figure 4.5: AI Chat to Human Handoff Implementation Evidence

4.4 Cross-Cutting Engineering Concerns

4.4.1 Validation, Security, and Error Boundaries

- Request validation is enforced at route boundary by Zod schemas and validation middleware.
- Protected routes use JWT middleware before service execution.
- Security headers are enabled by Helmet and payload compression by `compression`.
- Service-layer errors are propagated to controller-level response handling.

4.4.2 Observability and Operational Safety

- Winston daily rotation logs are configured for info and error channels.
- Reservation expiry subscriber triggers rollback for stale pending orders.
- Staff heartbeat expiry subscriber marks staff offline and triggers reassignment logic.
- Graceful shutdown ensures clean closure of HTTP, socket, Redis, and PostgreSQL resources.

4.5 Chapter Summary

This chapter presents the concrete backend realization of the proposed architecture, from API layering to reliability mechanisms such as atomic reservation, idempotent payment finalization, and chat continuity reassignment. The next chapter evaluates these implementations using functional and non-functional criteria.

Chapter 5

DISCUSSION AND EVALUATION

5.1 Evaluation Scope

This chapter evaluates how the implemented backend satisfies the objectives defined in Chapter 1 and the methodology in Chapter 3. The evaluation focuses on behavior-level correctness, consistency under contention, reliability of callback finalization, and continuity of realtime support flows.

5.1.1 Evaluation Targets

- Target 1: secure and correct authentication/session lifecycle.
- Target 2: stable catalog and cart behavior with stock-aware checks.
- Target 3: concurrency-safe checkout reservation for stock and voucher quotas.
- Target 4: reliable payment finalization with callback verification and idempotent handling.
- Target 5: realtime AI chat escalation and continuity when staff availability changes.

5.1.2 Evaluation Strategy

The evaluation uses scenario-based validation with representative request sequences:

- normal customer journey from login to paid order;
- concurrent checkout attempts on shared inventory;
- repeated and delayed payment callback events;
- chat escalation and reassignment on staff disconnect/heartbeat timeout.

5.2 Scenario-Based Results

5.2.1 Authentication and Session Security

The authentication module was validated across register, email verification, login, refresh, logout, and password recovery endpoints. The service-layer token checks and session invalidation logic provide consistent login-state transitions for protected APIs.

Table 5.1: Authentication Evaluation Outcomes

Scenario	Expected Behavior	Observed Outcome
Register and verify email	New account created and activated after verification	Verification flow completed and account state updated correctly
Refresh token rotation	Valid refresh token yields renewed access context	Token renewal accepted only with valid auth context
Logout + reuse token	Session/token should not remain reusable	Invalidated session state prevented unauthorized continuation
Password recovery (OTP)	Only valid OTP/reset context can change password	Reset flow accepted valid OTP context and rejected invalid cases

5.2.2 Catalog and Cart Behavior

Catalog endpoints returned structured product and filter payloads for browsing and detail views. Cart operations correctly handled add/update/remove transitions and kept cart snapshots synchronized with validation checks at service level.

Table 5.2: Catalog and Cart Evaluation Outcomes

Scenario	Expected Behavior	Observed Outcome
Product listing and detail retrieval	Correct pagination/filtering and detail payload	Responses aligned with requested filters and slug targets
Add to cart with valid variation	Item added or merged into active cart correctly	Cart update succeeded with snapshot recomputation
Quantity update beyond stock limit	Request should be blocked by stock validation	Validation rejected invalid quantity changes
Remove cart item	Cart state should remain consistent after deletion	Item removal completed and totals refreshed

5.2.3 Checkout Reservation Consistency

Checkout uses Redis and Lua to atomically reserve stock and discount capacity before order placement. This strategy addresses race conditions in high-contention scenarios by validating and applying reservation changes in a single atomic operation.

Table 5.3: Checkout Reservation Evaluation Outcomes

Scenario	Expected Behavior	Observed Outcome
Normal preview and place-order	Reservation created and pending order generated	Checkout succeeded with reservation metadata persisted
Concurrent reservation on same stock	No over-reservation should occur	Contended requests were bounded by atomic reservation checks
Reservation expiry without payment	Stale reservation should be rolled back	Expiry subscriber released resources and canceled stale order
Voucher quota contention	Usage limit should be preserved	Reservation counters prevented quota overflow during contention

5.2.4 Payment Callback Finalization

Payment processing was evaluated through MoMo callback scenarios covering valid, invalid, and repeated notifications. Signature verification and idempotent update strategy reduced the risk of duplicate or inconsistent finalization.

Table 5.4: Payment Finalization Evaluation Outcomes

Scenario	Expected Behavior	Observed Outcome
Valid callback signature	Order finalized according to payment result	Order/payment state updated and reservation handled correctly
Invalid callback signature	Callback must be rejected	Invalid callback processing path rejected untrusted payloads
Duplicate callback delivery	Final state should not be duplicated/corrupted	Idempotent guard prevented unsafe repeated finalization
Post-payment persistence	Order lines and discounts should be materialized	Order products and discount usage persisted for completed order

5.2.5 Realtime Chat and Handoff Continuity

The chat subsystem was evaluated with AI-first response handling, escalation requests, staff assignment, and disconnect recovery. Socket-based event flow and heartbeat-based reassignment preserved session continuity when staff availability changed.

Table 5.5: Realtime Chat Evaluation Outcomes

Scenario	Expected Behavior	Observed Outcome
AI-first customer interaction	Session receives AI response with context	AI response path executed for eligible messages
Manual escalation to human support	Session routed to available staff when possible	Assignment flow connected customer session to staff
Staff disconnect during active session	Session continuity should be maintained	Reassignment or fallback path kept session active
Heartbeat timeout event	Offline staff should be handled automatically	Expiry subscriber triggered offline and continuity workflow

5.3 Discussion of Engineering Trade-offs

5.3.1 Modular Monolith vs. Distributed Services

The modular monolith approach improved implementation speed and operational simplicity while preserving clear service boundaries. This is appropriate for the current project scale. However, the architecture still requires careful module discipline to avoid future coupling growth.

5.3.2 Redis Coordination vs. Pure Database Locking

Using Redis reservation counters and Lua scripts reduces lock contention and supports fast short-lived coordination. The trade-off is additional operational complexity (key lifecycle, rollback logic, expiry monitoring) compared with pure SQL transactions.

5.3.3 AI-first Support vs. Human-only Support

AI-first handling improves responsiveness for common requests and reduces staff load. Human handoff remains necessary for complex cases, and overall quality depends on routing quality, staff availability, and stable realtime signaling.

5.4 Limitations and Threats to Validity

- Quantitative load testing is constrained by available local infrastructure; large-scale production traffic patterns were not fully reproduced.
- External dependencies (payment provider and AI provider) introduce variable latency and failure modes that may affect end-to-end behavior.
- Some evaluation evidence is scenario-driven rather than long-term production telemetry.
- Multi-region deployment and disaster-recovery validation were outside the current project scope.

5.5 Chapter Summary

The evaluation indicates that the implemented backend satisfies the core objectives: secure session control, consistency-safe checkout reservation, reliable payment finalization, and resilient realtime support continuity. The main trade-off is increased coordination complexity, which can be further improved through deeper observability and larger-scale benchmarking in future work.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the design and implementation of a backend system for an intelligent fashion e-commerce platform. The implemented solution addresses six core domains: authentication and session security, catalog querying, cart lifecycle management, checkout reservation control, payment callback finalization, and real-time AI chat with human handoff.

From an engineering perspective, the backend applies a modular monolithic architecture with clear separation between routing, controller, service, and repository responsibilities. PostgreSQL is used as the transactional source of truth, while Redis provides short-lived coordination for reservation counters, heartbeat keys, and session-side runtime state. This combination enables practical reliability for concurrency-sensitive checkout and real-time support continuity.

6.2 Achievement of Objectives

The objectives defined in Chapter 1 were achieved as follows:

- **Modular backend architecture**: implemented through explicit API layering and bounded service responsibilities.
- **Consistency-safe checkout**: implemented by Redis and Lua-based atomic reservation for stock and discount quotas.
- **Payment callback correctness**: implemented with MoMo signature verification and idempotent finalization flow.
- **AI-to-human support continuity**: implemented with Socket.IO, staff assignment, heartbeat monitoring, and reassignment logic.
- **Operational robustness**: strengthened by validation middleware, structured logging, and graceful shutdown handling.

6.3 Limitations

Although the system meets the project goals, several limitations remain:

- Evaluation scale is constrained by available local test infrastructure and workload volume.
- Observability is currently log-centric; full distributed tracing and metrics dashboards are not yet fully integrated.
- External provider dependency (payment gateway and AI provider) can introduce latency variability and failure modes outside direct system control.
- Current deployment model is optimized for a modular monolith and has not yet been migrated to a full cloud-native multi-service architecture.

6.4 Future Work

Future development can focus on the following directions:

- Introduce queue-based asynchronous pipelines for heavy operations and burst smoothing.
- Expand performance testing with larger concurrent-user profiles and long-duration soak tests.
- Add centralized observability with metrics, tracing, and production-grade alert thresholds.
- Strengthen security posture through rate-limit tuning, abuse detection, and secret/key rotation automation.
- Extend recommendation and support intelligence with richer context retrieval and conversation quality evaluation.

6.5 Closing Remarks

The implemented backend demonstrates that combining transactional consistency controls with real-time support orchestration is feasible within a maintainable architecture for e-commerce systems. The current result provides a strong technical foundation for production hardening and future platform expansion.

Bibliography

[1] J. Smith and J. Doe, "Blockchain technology in education: A comprehensive review," *IEEE Transactions on Education*, vol. 45, no. 2, pp. 123–135, 2023.

[2] V. A. Nguyen and T. B. Tran, "A verifiable credential system for academic records," in *Proceedings of the 2024 ACM Conference on Computer Science Education*, (New York, NY, USA), pp. 456–461, ACM, 2024.

[3] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System.* Self-published, 2008.

[4] M. Sporny *et al.*, "Verifiable credentials data model 1.0," w3c recommendation, World Wide Web Consortium (W3C), 2019.

[5] Ethereum Foundation, "Ethereum documentation." `https://ethereum.org/en/developers/docs/`, 2024. Accessed: 2024-01-15.

[6] T. T. H. Le, "Blockchain-based transcript verification system," bachelor's thesis, The International University, Vietnam National University, 2023.

[7] W. Zhang, *Scalable Cryptographic Protocols for Distributed Systems.* PhD thesis, Massachusetts Institute of Technology, 2022.

Appendix A

Additional Technical Materials

A.1 API Endpoint Summary

This appendix can list endpoint groups used in the backend:

- /api/v1/auth
- /api/v1/catalog
- /api/v1/cart
- /api/v1/checkout
- /api/v1/payment
- /api/v1/chat

A.2 Redis Key Namespace

Recommended documentation table for Redis keys:

Table A.1: Redis Key Patterns Used in the Backend

Key Pattern	Purpose
reservation:{orderId}:items	Reserved stock snapshot per order
reservation:{orderId}:vouchers	Reserved voucher snapshot per order
reservation:{orderId}:ttl	Reservation expiration trigger
chat:staff:online	Online staff set for chat assignment
chat:staff:{staffId}:hb	Staff heartbeat TTL key
chat:session:{sessionUuid}:state	Chat mode/status cache

A.3 Additional Sequence Diagrams

Place extended sequence diagrams that are too detailed for the main chapters, for example:

- Reservation rollback after TTL expiration.
- Payment callback idempotency handling.
- Staff heartbeat expiration and chat reassignment.

A.4 Configuration Snapshot

This appendix can include a sanitized configuration example for reproducibility, without exposing secrets.